

Course: MATH 203

Dr. Ahmed Etman

Elliptic Curves Cryptography

Mazen Salem 202400447

Youssef Mohamed 202400956

Moustafa Hesham Sallam 202400624

Hassan Emad 202401299

Mahmoud Abdelwahab 202402046

The Introduction

With the rise of the internet we became more reliant on cryptography to keep our information secure. There are different ways to implement cryptography on our information but most of the implementations require a shared private key between the parties communicating. Sharing such a private key on a public channel has proven to be a complex problem, but many solutions to this problem have evolved over the years. One of the most popular ways to share private keys is with diffie hellman protocol. An extension to it is the Elliptic Curve Cryptography system [ECC]. ECC was first implemented in the mid of 1980 [1]. One of the main benefits of it is the relatively small key size and computational power needed for a great level of security. This paper aims to explain how ECC works and how it allows us to share private keys securely with relative ease.

The body

Cryptography

In any cryptography system to operate it must have a one way function[1]. That is a function that is easily computed but not easily reversed. This is useful in cryptography because we need to be able to generate an encryption and at the same time we want it to be hard for everyone else to trace back the original message without additional private information. This is true for every type of encryption and for any type of message, but there is one problem in order of the authentic receiver to be able to decrypt the message he needs to have a private key that is the same as the sender of the message sharing such key is the main problem ECC and diffie hellman solve.

Elliptic Curves

Elliptic Curves are curves that represent a relationship between x and y over a specific field and have some interesting properties. The general formula for defining an Elliptic Curve is $y^2 = x^3 + ax^2 + b$ (1) where a, b are constants. The values of a and b must follow the equation $4a^3 + 27b^2 \neq 0 \pmod{p}$. Moreover the choice of a and b determines the shape of the curve. Two examples of an Elliptic Curve are present in figure 1,2. Equation 1 represents Elliptic Curves over the field of all real numbers[2]. For our usage we will need to limit the values of the elliptic curve to a real number p . This is done by taking the modulus p of each side of equation 1. A useful property of elliptic curves is that each line will cross the curve two times if it is tangent to one point on it or three times if it is not tangent. The only exception is when the line is vertical then we assume the third point to be infinity or the natural point. Another useful property is that elliptic Curves are symmetric about the x -axis.

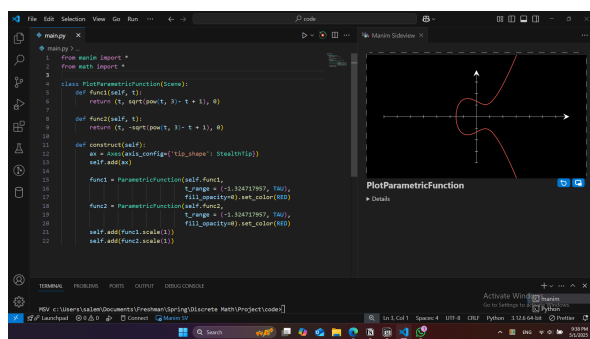


Figure 1

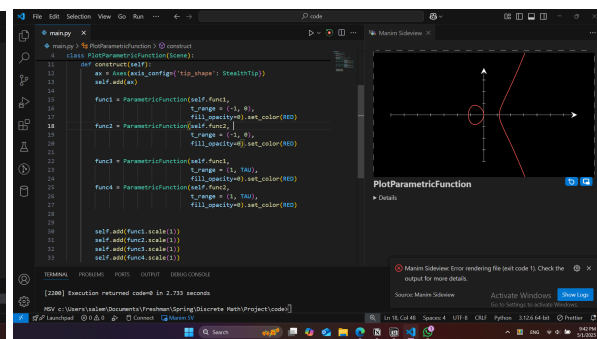


Figure 2

The Group Operations

As it turns out some operations can be done on an elliptic curve that can be used to make a one way function. These operations are addition and scalar multiplication. Any two points contained inside the elliptic curve can be summed and the resulting point is the inverse of the point generated by crossing the curve with the line connecting the two points summed[3]. Scalar Multiplication is simpler but rather than adding two points we add a point to itself multiple times. The resulting point will be the inverse of the point generated by crossing the tangent of the point with the curve. Note that in either operation the resulting point lies on the same curve. To compute the resulting point from each operation we first need to calculate s which is the slope of the line connecting the two points in addition or the slope of tangent in scalar multiplication

$$s = \frac{y_q - y_p}{x_q - x_p} \qquad s = \frac{3(x_p)^2 + a}{2y_p}$$

Getting the values of the resulting point is easily computed as follows

$$x_R = s^2 - 2x_p \qquad y_R = s(x_p - x_R) - y_p$$

These computations can be used in encryption because the addition operation is not easily reversible. That is if we know a point P and another point Q and we know that $P = kQ$ we still can't calculate k . And this is the discrete log problem.

Diffie Hellman Protocol

Diffie hellman protocol is simply a way to ensure two participants have the same private key without directly sharing it or sharing any information that would allow any unwanted party to know the private key. Normal Diffie Hellman uses exponential arithmetic to ensure such a thing, but we will be using EEC as it will allow us to use much smaller key values for better security. The domain parameters of such a system are $\{p, a, b, G, n, h\}$ [4]. P is used to define the

field on which the curve is drawn. a and b are used to define the curve itself. G is the base point group. n is the size of the cycle group that starts from the base point (B) and contains multiples of B until reaching B again. h is a cofactor and it equals to all the points in the field divided by the points in the group G . Note ideally the value of h will be close to 1. If it was greater than 4, then our encryption wouldn't be secure. Implementing Diffie Hellman Protocol is as follows, First each participant will have a private number a , b respectively. Second each participant will calculate a point A , B respectively as $A = aG$, $B = bG$ [G is the base point]. Then we can send A , B in a public channel knowing that no one will be able to generate a or b from it. When each participant gets the point from the other they multiply it with their own private key which makes each participant have the point abG . This point is private no one except the two participants know about, and anyone eavesdropping will not be able to calculate it as well. Note that if ab is greater than the number of elements of G we take mod n .

Applications

Crypto Systems Based on Elliptic Curves:

Algorithms of cryptosystems based on elliptic curves use tuples of parameters to define an underlying curve E . $E=(p, a, b, P, n, h)$ where p is a prime number which determines the finite prime field, a and b are coefficients of the curve, P – point on the curve, generator of cyclic subgroup, n – order of subgroup, h – cofactor of subgroup [5, 12].

As mentioned, there are classes of elliptic curves which are considered unsafe for use in cryptography. For instance, there are weak elliptic curves which allow calculation of discrete logarithms in polynomial time [11, 12]. To prevent malicious use of elliptic curves in the implementations of crypto systems, the curve coefficients a ; b and the

subgroup generator P can be randomized by using hash functions. In such cases, the set of curve parameters is extended by adding probabilistic number s , which is used to generate parameters $a; b$ and P . An elliptic curve with probabilistic parameter s is denoted as a verifiably random elliptic curve [12]. The generation of verifiably random curves is standardized, there are different methods of selecting the most appropriate hash function to generate the curve.

Although such approach provides relative protection against maliciously selected elliptic curves, however, there are certain risks involved, such as security and reliability of hash functions being used to generate the curve parameters.

General key generation in elliptic curve based crypto systems: For an elliptic curve $E = (p; a; b; P; n; h)$, there are key pairs $(d; Q)$, where d is the private key which is natural number from the interval $[1; n-1]$ and Q is the point of an elliptic curve which is the public key $Q = (x_q; y_q) = dP$ [5].

For simplicity, we shall restrict our attention to elliptic curves over Z_p , where p is a prime greater than 3. We mention, however, that elliptic curves can more generally be defined over any finite field [4]. An elliptic curve E over Z_p is defined by an equation of the form.

$$y^2 = x^3 + ax + b$$

where $a, b \in Z_p$, and $\neq 0 \pmod{p}$, together with a special point O called the $4a^3 + 27b^3$ point at infinity. The set $E(Z_p)$ consists of all points (x, y) , $x \in Z_p$, $y \in Z_p$, which satisfy the defining equation (1), together with O .

ECDH Crypto System:

ECDH (Elliptic Curve Diffie-Hellman) is the version of Diffie-Hellman key exchange

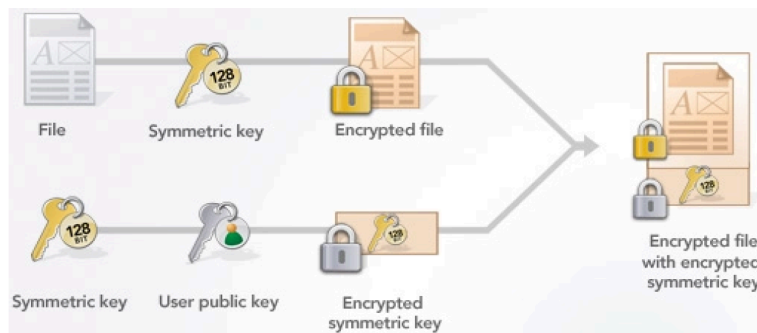
algorithm for elliptic curves, which determines the method how two communication participants A and B can generate key pairs and exchange their public keys via insecure channels. The algorithm determines only the method how key pairs are generated, the user defines the relation between encryption keys and data to be encrypted [7, 12]. After the keys have been exchanged it is common to use symmetric encryption methods. In practice, ECDH crypto system can be successfully adapted to different data security solutions [7].

The ECDH algorithm: Given an elliptic curve over finite prime field $E(\mathbb{F}_p)$. Communication participants A and B agree on a point $Q \in E(\mathbb{F}_p)$, which is freely available in the communication medium. The participant A secretly chooses probabilistic positive integer k_A ; calculates $Q_A = k_A Q$ and send it to party B. The member B also selects k_B probabilistic positive integer, calculates $Q_B = k_B Q$ and send it to participant A. The shared secret is $P = k_A k_B Q$. The participant A calculates P, by multiplying the received point Q_B with the secret private key k_A . The participant B respectively calculates P by multiplying the received Q_A point with the secret private key k_B [12].

After the public key exchange, one of the most typical methods how the encrypted messages are synchronized is the version of ElGamal crypto system for elliptic curves [1]. As the set $E(\mathbb{F}_p)$ is finite there is a limited set of information which could be encrypted by using points of the curve [1]. Also, there are no practical methods available to deterministically generate points of the given curve [1].

The problem can be solved by creating relation between the x-coordinate of given curve point $P \in E(\mathbb{F}_p)$ and a field $\mathbb{F}_p$ element that may not be a point on the curve [1]. There are algorithms Compress Point and Decompress Point available which allow reduction of the amount of memory required for storing the curve points in the computer

memory by two times. It is possible to clearly identify the curve point $P = (xy)$ by specifying the x-coordinate of point P and parity bit b, $b \equiv y \pmod{2}$ [1]. Assume there is a fixed secret unavailable private key FPK, and public key FP for it is available to the application. As a user enters a value, a new private key is generated, then a shared key is produced from exchanging it from FP. Resulting sharing key is different every time. Symmetric encryption key DSK is derived from a shared key using KDF (Scrypt or HKDF).



Bitcoin:

ECC is used for a variant to derive public keys and sign Bitcoin transactions. To have a high level of security and efficiency. Bitcoin uses the Elliptic Curve, “secp256k1”, a takes the value

of 0, and b takes the value of 7. As a result, the equation is as follows: $y^2 = x^3 + 7$. That

makes it easy in one direction (derive the Public Key) and technically impossible to go the other way, ensuring the robustness of the system. Moreover, the Public Key comes from the Private Key and a constant G known as the Generator Point, which is static and defined in the secp256k1 curve parameters, through the following operation:

$$K \text{ (Public Key)} = k \text{ (Private Key)} \times G \text{ (Generator Point)}$$

This operation is performed on the coordinates of the Elliptic Curve, instead of real numbers. So, in this particular case, multiplying is best interpreted as adding the Generator Point as many times as indicated by the private key value. If the Private Key were «5», to obtain the Public Key, you would have to add $G+G+G+G+G = 5G$. the Generator Point (G) is a constant already determined in the initial parameters of the secp256k1 elliptic curve: 0479BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448 A6855419 9C47D08F FB10D4B8

CONCLUSION

Through the innovative technologies of Elliptic Curve Cryptography, the internet has become more secure than ever. ECC marks a significant advancement in the field of secure communication and cryptography as its usage of mathematical properties of elliptic curves makes it more robust than traditional methods like RSA. In addition, ECC can be easily implemented with limited resources as it doesn't require a big private key making it a very suitable option for mobile devices and embedded systems. The usage of ECC with the Diffie-Hellman protocol can ensure a safe sharing of private keys as long as the Discrete Logarithm Problem stays unsolved.

Works Cited

[1]R. Pierce, “Elliptic Curve Diffie Hellman,” *YouTube*. Dec. 10, 2014. [YouTube Video].

Available: <https://www.youtube.com/watch?v=F3zzNa42-tQ>

[2]“Elliptic Curve Cryptography Overview,” *YouTube*. Oct. 14, 2015. Accessed: Dec. 03, 2019.

[YouTube Video]. Available: <https://www.youtube.com/watch?v=dCvB-mhkT0w>

[3]“Elliptic Curves - Computerphile,” *YouTube*. Jan. 16, 2018. Accessed: Dec. 17, 2019.

[YouTube Video]. Available: <https://www.youtube.com/watch?v=NF1pwjL9-DE>

[4]N. Koblitz, A. Menezes, and S. Vanstone, *Designs, Codes and Cryptography*, vol. 19, no. 2/3,

pp. 173–193, 2000, doi: <https://doi.org/10.1023/a:1008354106356>.

[5]Kris Magons, “Applications and Benefits of Elliptic Curve Cryptography,” pp. 32–42, Jan. 2016.